Alunos: Ozaeltton Matheus da Silva Medeiros (20231038060022) e Kauê Adriano dos Santos Dantas (20241038060007)

Agrupamento (Clusterização) de dados - vgsales

Código-fonte: K-Means, Agrupamento Hierárquico Aglomerativo e Gaussian Mixture (GMM), com comparação dos grupos descobertos com a rotulação existente (coluna-alvo Mercado).

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.mixture import GaussianMixture
from sklearn.metrics import (silhouette_score, davies_bouldin_score,
                                                    adjusted_rand_score,
normalized_mutual_info_score,
                             homogeneity_completeness_v_measure)

# Parâmetros da variável-alvo
# Participação do Japão nas vendas (NA + EU + JP) de cada jogo:
# abaixo de LIMIAR_OCIDENTE -> "Ocidente"
# acima de LIMIAR_ORIENTE   -> "Oriente"
# entre os dois             -> "Equilibrado"
LIMIAR_OCIDENTE = 0.10
LIMIAR_ORIENTE = 0.50

# Parâmetros do agrupamento
SEED = 42
N_COMPONENTES_PCA = 10        # componentes mantidas após o PCA
K_MIN, K_MAX = 2, 8             # intervalo de k testado para escolher o
n° de grupos
N_AMOSTRA_HIERARQUICO  =  5000    # o aglomerativo guarda a matriz de
distâncias (memória
                                # cresce com n²), por isso roda em uma
amostra do dataset

data = pd.read_csv('./vgsales.csv')
```

```
print("-" * 20 + "Primeiras linhas do dataset" + "-" * 20)
print(data.head())

print("-" * 20 + "Últimas linhas do dataset" + "-" * 20)
print(data.tail())

print("-" * 20 + "Formato do dataset" + "-" * 20)
print(data.shape)  # esperado: cerca de 16.598 linhas e 11 colunas

print("-" * 20 + "Tipos das colunas" + "-" * 20)
print(data.dtypes)

# Valores nulos por coluna (o pandas já lê "N/A" do CSV como nulo).
# Publisher e Year possuem nulos; as colunas de vendas não.
print("-" * 20 + "Valores nulos por coluna" + "-" * 20)
print(data.isnull().sum())

# Linhas duplicadas
print("-" * 20 + "Linhas duplicadas" + "-" * 20)
print(data.duplicated().sum())

# Os valores 0 nas colunas de vendas têm significado para a análise (o
jogo
#  vendeu  menos  de  10  mil  cópias  na  região,  no  arredondamento  do
dataset),
# portanto não são apagados nem substituídos.

# Exclusão de colunas desnecessárias para a análise:
# Rank: é calculado a partir de Global_Sales, que contém as vendas das
# regiões usadas para criar o alvo (vazamento de informação).
# Global_Sales: soma das regiões, incluindo as que usamos para o alvo.
# Other_Sales: fora do recorte Ocidente x Oriente.
#  Name:  identificador  (milhares  de  valores  únicos),  sem  poder  de
generalização.
#  A  coluna  Year  é  MANTIDA:  plataformas  e  gêneros  mudam  muito  com  a
época,
# e o ano ajuda a separar o efeito temporal do efeito cultural.
data      =      data.drop(columns=['Rank',      'Name',      'Other_Sales',
'Global_Sales'])

# Ocidente = América do Norte + Europa; Oriente = Japão.
data["West_Sales"] = data["NA_Sales"] + data["EU_Sales"]
```

```
data["East_Sales"] = data["JP_Sales"]
total_regioes = data["West_Sales"] + data["East_Sales"]

# Jogos sem vendas nas três regiões (vendas apenas em Other_Sales) não têm
# como receber uma classe e são removidos.
sem_vendas = total_regioes == 0
print("-" * 20 + "Jogos removidos (sem vendas em NA, EU e JP)" + "-" * 20)
print(int(sem_vendas.sum()))
data = data[~sem_vendas].copy()
total_regioes = total_regioes[~sem_vendas]

# Comparar West_Sales com East_Sales diretamente favoreceria o Ocidente por
# construção (duas regiões somadas contra uma). Por isso o alvo usa a
# participação do Japão nas vendas de cada jogo: mede se o jogo performa de
# forma relativamente melhor no Oriente, independentemente do tamanho do mercado.
data["Participacao_Oriente"] = data["East_Sales"] / total_regioes

# Construção da variável-alvo (Mercado) a partir da participação do Japão nas vendas.
# Indica qual mercado o jogo vende mais: Ocidente, Oriente ou Equilibrado.
data["Mercado"] = np.select(
    [
        data["Participacao_Oriente"] < LIMIAR_OCIDENTE,
        data["Participacao_Oriente"] > LIMIAR_ORIENTE,
    ],
    ["Ocidente", "Oriente"],
    default="Equilibrado",
)

print("-" * 20 + "Distribuição da variável-alvo (Mercado)" + "-" * 20)
print(data["Mercado"].value_counts())
print(data["Mercado"].value_counts(normalize=True).round(3))
# Classes desbalanceadas: nas próximas etapas usar validação cruzada
# estratificada e métricas como F1 macro / acurácia balanceada.

# As colunas de vendas e derivadas saem das características, evitando vazamento de informação.
```

```
colunas_vendas = ["NA_Sales", "EU_Sales", "JP_Sales",
                  "West_Sales", "East_Sales", "Participacao_Oriente"]

y = data["Mercado"]
X = data.drop(columns=colunas_vendas + ["Mercado"])

colunas_categoricas = ['Platform', 'Genre', 'Publisher']
colunas_numericas = ['Year']

# Pré-processamento abaixo
# Categóricas: nulos viram "Unknown"; categorias raras (poucos jogos, comum em
# Publisher) são agrupadas em uma categoria "infrequente" para evitar centenas
# de colunas quase vazias.
pipeline_categorico = Pipeline([
                    ('imputer',       SimpleImputer(strategy='constant', fill_value='Unknown')),
        ('onehot',   OneHotEncoder(handle_unknown='infrequent_if_exist', min_frequency=10)),
])

# Numéricas: nulos viram a mediana; depois padronização.
pipeline_numerico = Pipeline([
   ('imputer', SimpleImputer(strategy='median')),
   ('scaler', StandardScaler()),
])

# Este "preprocessor" pode ser usado dentro de um Pipeline com o classificador
# nas próximas etapas, para ser reajustado a cada dobra da validação cruzada.
preprocessor = ColumnTransformer(
   transformers=[
       ('cat', pipeline_categorico, colunas_categoricas),
       ('num', pipeline_numerico, colunas_numericas),
   ]
)

# Agrupamento (clusterização)
# O agrupamento é não supervisionado: usa todos os jogos (sem divisão treino/teste)
```

```
# e a coluna-alvo (Mercado) NÃO participa da formação dos grupos. Ela é usada
# apenas no final, para comparar os grupos descobertos com a rotulação existente.
X_proc = preprocessor.fit_transform(X)
if hasattr(X_proc, "toarray"):    # o one-hot pode devolver matriz esparsa
    X_proc = X_proc.toarray()

print("-" * 20 + "Resultado do pré-processamento" + "-" * 20)
print("Dimensão:", X_proc.shape)

# O PCA reduz as colunas geradas pelo one-hot, deixando as distâncias mais estáveis
pca = PCA(n_components=min(N_COMPONENTES_PCA, X_proc.shape[1]), random_state=SEED)
X_pca = pca.fit_transform(X_proc)
print(f"Variância explicada pelo PCA: {pca.explained_variance_ratio_.sum():.2%}")

# Escolha do número de grupos (K-Means): método do cotovelo e silhouette
inercias = {}
silhouettes = {}
for k in range(K_MIN, K_MAX + 1):
    kmeans = KMeans(n_clusters=k, n_init=10, random_state=SEED).fit(X_pca)
    inercias[k] = kmeans.inertia_
    silhouettes[k] = silhouette_score(X_pca, kmeans.labels_,
                                      sample_size=5000, random_state=SEED)

k_silhouette = max(silhouettes, key=silhouettes.get)
k_alvo = y.nunique()    # nº de grupos igual ao nº de classes da variável-alvo

print("-" * 20 + "Escolha do número de grupos" + "-" * 20)
print("Silhouette por k:", {k: round(v, 3) for k, v in silhouettes.items()})
print("k com maior silhouette:", k_silhouette)
print("k igual ao nº de classes de Mercado:", k_alvo)

fig, eixos = plt.subplots(1, 2, figsize=(10, 4))
```

```
eixos[0].plot(list(inercias.keys()),           list(inercias.values()),
marker='o')
eixos[0].set_title("Método do cotovelo (K-Means)")
eixos[0].set_xlabel("k")
eixos[0].set_ylabel("Inércia")
eixos[1].plot(list(silhouettes.keys()),      list(silhouettes.values()),
marker='o', color='tab:orange')
eixos[1].set_title("Silhouette (K-Means)")
eixos[1].set_xlabel("k")
eixos[1].set_ylabel("Silhouette")
plt.tight_layout()
plt.savefig("escolha_k.png", dpi=150)
plt.show()

# Amostra para o agrupamento hierárquico (os demais algoritmos usam
todos os jogos)
todos = np.arange(len(X_pca))
amostra = np.sort(np.random.RandomState(SEED).choice(
   todos, size=min(N_AMOSTRA_HIERARQUICO, len(todos)), replace=False))


def aplicar_algoritmos(k):
   """Executa os três algoritmos com k grupos. Devolve {nome: (índices
usados, grupos)}."""
   return {
               "K-Means":   (todos,   KMeans(n_clusters=k,   n_init=10,
random_state=SEED).fit_predict(X_pca)),
      "Aglomerativo (Ward)": (amostra, AgglomerativeClustering(
          n_clusters=k, linkage="ward").fit_predict(X_pca[amostra])),
                    "GMM":    (todos,    GaussianMixture(n_components=k,
covariance_type="full",
                                                        reg_covar=1e-4,
random_state=SEED).fit_predict(X_pca)),
   }


def pureza(grupos, rotulo):
    """Fração de jogos que pertencem à classe majoritária do seu
grupo."""
   tabela = pd.crosstab(grupos, rotulo)
   return tabela.max(axis=1).sum() / tabela.values.sum()
```

```
def avaliar(nome, k, indices, grupos):
    """Métricas internas (qualidade dos grupos) e externas (concordância
com Mercado)."""
    rotulo = y.iloc[indices]
                  homogeneidade,       completude,       v_measure       =
homogeneity_completeness_v_measure(rotulo, grupos)
    return {
        "Algoritmo": nome,
        "k": k,
        "Jogos": len(indices),
        "Silhouette": silhouette_score(X_pca[indices], grupos,
                                                          sample_size=5000,
random_state=SEED),
        "Davies-Bouldin": davies_bouldin_score(X_pca[indices], grupos),
        "ARI": adjusted_rand_score(rotulo, grupos),
        "NMI": normalized_mutual_info_score(rotulo, grupos),
        "Homogeneidade": homogeneidade,
        "Completude": completude,
        "V-measure": v_measure,
        "Pureza": pureza(grupos, rotulo),
    }


resultados = []
grupos_por_k = {}
for k in sorted({k_alvo, k_silhouette}):
    grupos_por_k[k] = aplicar_algoritmos(k)
    for nome, (indices, grupos) in grupos_por_k[k].items():
        resultados.append(avaliar(nome, k, indices, grupos))

comparativo = pd.DataFrame(resultados).round(3)

# Referência para a pureza: sempre prever a classe majoritária de
Mercado
print("-" * 20 + "Pureza de referência (classe majoritária de Mercado)"
+ "-" * 20)
print(round(y.value_counts(normalize=True).max(), 3))

pd.set_option("display.width", 200)
pd.set_option("display.max_columns", None)
print("-" * 20 + "Comparativo entre os algoritmos" + "-" * 20)
print(comparativo.to_string(index=False))
```

```
# Comparação detalhada dos grupos com a rotulação existente (k = nº de
classes)
for nome, (indices, grupos) in grupos_por_k[k_alvo].items():
   print("-" * 20 + f"{nome}: grupos x Mercado" + "-" * 20)
                print(pd.crosstab(pd.Series(grupos,      name="Grupo"),
y.iloc[indices].values,
                    rownames=["Grupo"], colnames=["Mercado"]))

    # Perfil de cada grupo (vendas em milhões, participação do Japão e
ano médio)
         perfil    =    data.iloc[indices][["West_Sales",    "East_Sales",
"Participacao_Oriente", "Year"]]
   perfil = perfil.groupby(grupos).mean().round(3)
   perfil["Jogos"] = pd.Series(grupos).value_counts().sort_index()
   print("Perfil médio de cada grupo:")
   print(perfil)

   # Categoria mais frequente de cada grupo
   for coluna in colunas_categoricas:
       print(f"{coluna} mais frequente por grupo:")
       print(X.iloc[indices][coluna].groupby(grupos).agg(
           lambda s: s.value_counts().index[0]))

# Visualização: projeção nas duas primeiras componentes do PCA
paineis = [("Mercado (rotulação existente)", y.values, todos)]
for nome, (indices, grupos) in grupos_por_k[k_alvo].items():
   paineis.append((nome, grupos, indices))

fig, eixos = plt.subplots(2, 2, figsize=(12, 9))
for eixo, (titulo, rotulos, indices) in zip(eixos.ravel(), paineis):
   categorias = pd.Series(rotulos).astype(str)
   for categoria in sorted(categorias.unique()):
       mascara = (categorias == categoria).values
        eixo.scatter(X_pca[indices][mascara, 0], X_pca[indices][mascara,
1],
                    s=8, alpha=0.5, label=categoria)
   eixo.set_title(titulo)
   eixo.set_xlabel("PC1")
   eixo.set_ylabel("PC2")
   eixo.legend(fontsize=8)
plt.tight_layout()
plt.savefig("comparacao_grupos.png", dpi=150)
plt.show()
```